FAU Erlangen-Nürnberg

Department Germanistik & Komparatistik

Lehrstuhl für Korpus- und Computerlinguistik

Hauptseminar Deep Learning und Künstliche Intelligenz in der maschinellen Sprachverarbeitung

Sommersemester 2023

Prof. Dr. Stephanie Evert, Michael Ruppert

Teilprojekt 3:
Labelerkennung mit Named-Entity Recognition Ansatz

Xinyao Lu

Matrikelnummer: 23008498

xinyao.lu@fau.de

Computerlinguistik/Germanistik

5. Fachsemester

Abgabe: 06.03.2024

Inhaltsverzeichnis

1 Allgemeine Beschreibung des Teilprojekts **4**

2 Vorverarbeitung des Datensatzes für Labelerkennung **5**
2.1 Tokenisierung 5
2.2 Zuordnung der Labels 5
2.3 Adjudikation 6

3 Vortrainiertes Embedding-Modell und Evaluation **7**
3.1 Verwendung des BERT-Embeddings 7
3.2 Evaluation 7

4 Modellarchitektur **8**
4.1 Optimierung der Klassengewichtung 9
4.2 Experimenten mit verschiedenen Modellarchitekturen 10
4.3 Fazit 12

Tabellenverzeichnis

1 Beispiel zur lockeren Zuordnung (Recht1) 5
2 Zuordnungsabfolge 6
3 Zuordnungsergebnisse 6
4 Anzahl der Geschichten im vollständgen und intensivierten Datensatz 8
5 Menge der „Insider“-Labels im Testingsdatensatz 9
6 Varianten von Klassengewichtungen 9
7 Grundarchitektur 10
8 Hidden-Layer 11
9 Spezifisches Ergebnis des Modells mit einem GRU Hidden Layer 12

Abbildungsverzeichnis

1 Evaluation unterschiedlicher Klassengewichtungen 10
2 Vergleich verschiedener Hidden-Layer (Evaluation auf der Token-Ebene) 11
3 weight1: bester Zustand nach 16 Epochen 14
4 weight5: bester Zustand nach 10 Epochen 14
5 weight10: bester Zustand nach 11 Epochen 14
6 weight15: bester Zustand nach 6 Epochen 15
7 weight20: bester Zustand nach 5 Epochen 15
8 weight-balanced: (26.7) bester Zustand nach 6 Epochen 15
9 weight30: bester Zustand nach 8 Epochen 15
10 SimpleDense: bester Zustand nach 49 Epochen 16
11 BLSTM-60: bester Zustand nach 13 Epochen 16
12 BLSTM-173: bester Zustand nach 10 Epochen 16
13 BGRU-173: bester Zustand nach 12 Epochen 17
14 BLSTM+LSTM: bester Zustand nach 10 Epochen 17

15 BGRU+GRU: bester Zustand nach 14 Epochen 17
16 Überanpassung verschiedener Modelle 18

1 Allgemeine Beschreibung des Teilprojekts

Im *Teilprojekt 3: Labelerkennung mit Named-Entity Recognition Ansatz* wird es versucht, mit Hilfe des nSex-Datensatzes die auf sexuelle Motivation hindeutenden Ausdrücke automatisch zu erkennen, d. h. ein Sprachmodell zu entwickeln, das mit den manuell erstellten Annotationen im Datensatz *trainiert* wird und dazu dient, mit dem gleichen Maßstab sexuell motivierte Ausdrücke in den fremden Texten herauszufinden. Im idealsten Falle könnte das Modell als ein Programm für die automatische Annotation benutzt werden. Um dieses Ziel zu erreichen, wird es in diesem Teilprojekt ein *Label-Token-Zuordnung* Ansatz verwendet, im nach der Tokenisierung jedem Token ein Label zugeordnet wird. Die von den Annotationen markierten Tokens bekommen ein *Insider*-Label. Alle anderen Tokens bekommen ein *Outsider*-Label. Die Aufgabe des Sprachmodells ist dann das richtige Label für jedes Token vorherzusagen.

Es gibt hauptsächlich zwei Unterschiede zwischen diesem Teilprojekt und dem *Teilprojekt 1: Maschinelle Klassifikation von kurzen Textsegmenten*: Erstens übernimmt das Sprachmodell in diesem Teilprojekt als Eingabedaten nicht Textsegmente, sondern vollständige Texte. Zweitens hat dieses Teilprojekt zwei Ziele: Es versucht sowohl die Art der Labels (**Welche** Labels gibt es im Text?), als auch die Positionen der Labels (**Wo** sind die Labels?) zu erkennen. Weil mehr als die Hälfte der Subkategorien weniger als hundert Mal im gesamten nSex-Datensatz vorkommt, (vgl. Teilprojekt 1, *Abbildung 5: Verteilung der Unterkategorien im nSex Datensatz*) versucht das Modell hier die 8 Hauptkategorien zu erkennen.[1] Zusammenfassend kann man sagen, dass nach der Zielsetzung Teilprojekt 3 eine Fortsetzung des Teilprojekts 1 ist, allerdings nach der Herangehensweise die beiden Teilprojekte unterschiedliche Ansätze verwenden: Teilprojekt 1 ist eine Multilabel-Klassifikation, während Teilprojekt 3 einen NER (Named-Entity Recognition) Ansatz verwendet.

Es gibt vor allem zwei Herausforderungen für dieses Teilprojekt. Erstens ist die Vielfältigkeit der sexuell motivierten Ausdrücke in einem relativ kleinen Datensatz. Um dieses Problem umzugehen, wird vortrainiertes Embedding-Modell als Embedding-Layer eingesetzt, damit das Modell nicht ausschließlich die gleichen Ausdrücke, sondern auch relevante Ausdrücke erkennen kann. Die zweite Herausforderung ist die unbestimmte Länge der Ausdrücke. Dieses Problem hat auch einen wesentlichen Einfluss im Teilprojekt 1, wobei neuronale Netzwerke sich bei längeren Textsegmenten deulich besser als traditionelle Lernverfahren verhalten. In diesem Teilprojekt wird es angenommen, dass alle in den originalen Texten eingebetteten Annotationen sich auf dem letzten Token beziehen. Allen anderen Tokens werden ein *Outsider*-Label zugeteilt. Darüber hinaus werden in den Experimenten vor allem Modellarchitekturen mit rekurrentem neuronalem Netz (RNN), die Informationen aus größerem Kontext berücksichtigen können, wie LSTM (Long Short-Term Memory) und GRU (Gated Recurrent Unit), ausgewählt. Als Grundlage der Neuronalen-Netzwerke werden Resourcen aus der *Keras* Deep-Learning-Bibliothek benutzt.

[1] Häufigkeiten der Hauptkategorien im gesamten Datensatz: Instrumentelle: 6858, Sonstige: 4390, Zielerreichung: 1884, Erfolgserwartung: 1494, Bedürfnis: 1136, Affektlage: 1209, Hindernisse: 430, Misserfolgserwartung: 158

2 Vorverarbeitung des Datensatzes für Labelerkennung

Um möglich mehr Annotationen in den Datensatz einzubeziehen, werden alle Texte aus den sechs Studien gleichartig behandelt. d. h. es wird angenommen, dass es für dieses Teilprojekt keine grundsätzliche Rolle spielt, ob ein Text vor oder nach dem *Priming* geschrieben ist.

2.1 Tokenisierung

Die Datenvorverarbeitung des Teamprojekts wandelt die in Word-Dateien gespeicherten originalen Texte in XML-Format um. Formate der in den Texten eingebetteten Annotationen werden vereinheitlicht. Kodierungsfehler sind korrigiert. Auf dieser Grundlage werden die vorverarbeiteten Texte mit dem Tokenizer *SoMaJo* in Wörter und Satzzeichen zerlegt. („Tokenisierung“) *SoMaJo* ist ein Tokenizer für deutsche und englische Texte, der sich auf Texte aus sozialen Medien fokusiert.[2] Der Tokenizer *SoMaJo* wird ausgewählt, weil die während des Experiments in beschränkter Zeit geschriebenen Texte viele Ähnlichkeiten mit Texten aus sozialen Medien aufweisen. Ein *span-Tokenizer* wird als Erweiterung für *SoMaJo* implementiert, um die Position jedes Tokens in den originalen Texten aufzuzeichnen. Die Positionen der in den Texten eingebetteten Annotationen wurden auch aufgezeichnet, bevor sie vor der Tokenisierung aus den Texten gelöscht werden. Überflüssige Leerzeichen werden vor der Tokenisierung entfernt, damit sie die Aufzeichnungen des *span-Tokenizers* nicht beeinträchtigen.

2.2 Zuordnung der Labels

Nach der Aufzeichnungen der *span-Tokenizer* wird die Label-Token-Zuordnung vorgenommen. Allerdings ist eine 1:1 Zuordnung nicht immer möglich. Weil wiederholte Satzzeichen und andere Tippfehler manchmal von *SoMaJo* automatisch entfernt werden, könnte es zu einer Abweichung von wenigen Buchstaben in der Aufzeichnung des *span-Tokenizers* führen. Um die Auswirkung solcher Abweichungen abzuschwächen, wird eine lockere Zuordnung implementiert. Das bedeutet, wenn die aufgezeichnete Startposition von einer Annotation nicht unmittelbar der Endposition des letzten Tokens folgt, wird eine Zuordnung trotzdem vorgenommen. Folgendes ist ein Beispiel zu einer lockeren Zuordnung:

Originaler Text	„Beide fühlen sich einander hingezogen [SexAnz] ... “
Tokens	[Beide, fühlen, sich, einander, hingezogen]
Spans	[(0, 5), (6, 12), (13, 17), (18, 26), (27, **37**)]
Position des Annotation	(**39**, 47)

Token	Beide	fühlen	sich	einander	hingezogen
Label	Outsider	Outsider	Outsider	Outsider	I-SexAnz

Tabelle 1: Beispiel zur lockeren Zuordnung (Recht1)

[2]Beschreibung von SoMaJo: „SoMaJo is a rule-based tokenizer and sentence splitter that implements tokenization guidelines for German and English. It has a strong focus on web and social media texts ... “ (https://github.com/tsproisl/SoMaJo)

In diesem Beispiel behindern die doppelten Leerzeichen zwischen dem Token „hingezogen“ und dem Label „[SexAnz]“ eine direkte Zuordnung. In diesem Fall versucht das Programm nach der folgenden Abfolge, das Label dem nächstgelegenen Token zuzuordnen:

Abfolge	Typ	Endposition des Tokens	Startpostion des labels
1	Exakt	n	n+1
2	Recht1	n	n+2
3	Link1	n	n
4	Recht2	n	n+3
5	Link2	n	n-1

Tabelle 2: Zuordnungsabfolge

Eine maximale Abweichung bis zu zwei Buchstaben an beiden Seiten wird zugelassen. Wenn ein Label trotzdem nicht zugeordnet werden kann, scheitert dann die Zuordnung. Jedes Label wird nur einmal zugeordnet. Tabelle 3 zeigt die Ergebnisse der Label-Token-Zuordnung bei jeden Studien. Fast 95% aller Labels aus den sechs Studien können mit kleiner Anpassung dem vorherigen Token zugeordnet werden. Der 5% Verlust lässt sich zwar nicht vernachlässigen, aber soll keine wesentliche Auswirkung auf die Qualität des Datensatzes haben.

Studie	Exakt	Recht1	Link1	Recht2	Link2	Erfolgsrate
Studie1	3,3%	32,7%	2,8%	57,9%	0,6%	**97,4%**
Studie2	4,0%	29,4%	4,0%	56,2%	1,3%	**94,8%**
Studie3	1,8%	30,3%	1,0%	62,8%	0,7%	**96,6%**
Studie4	0,5%	34,4%	0,1%	64,9%	0,0%	**99,8%**
Studie5	7,3%	28,3%	6,0%	45,7%	2,9%	**90,2%**
Studie6	3,0%	29,2%	2,4%	55,2%	1,6%	**91,4%**

Tabelle 3: Zuordnungsergebnisse

2.3 Adjudikation

Eine automatische Adjudikation bei den mehrfach annotierten Texten wird implementiert. Bei der Adjudikation werden Konflikte zwischen unterschiedlichen Annotationen aufgelöst.[3] Die automatische Adjudikation in diesem Teilprojekt wird auch mit dem Ziel gestaltet, um so viele Annotationen wie möglich einzubeziehen: Bei jeder Studie wird ein Annotator als *Autorität* festgelegt. Wenn die Annotatoren unterschiedliche Labels an gleicher Stelle erteilen, wird immer die Annotation des autoritären Annotators in den Datensatz übernommen. Erteilt der autoritäre Annotator an einer Stelle keine Annotation, werden dann die Annotationen der anderen Annotatoren als Ergänzung übernommen.

[3] Über die Aufgabeteilung zwischen den Annotatoren s. Teilprojekt 1, *Abbildung 3: Aufteilung des nSex Datensatzes in drei Teildatensätze*

3 Vortrainiertes Embedding-Modell und Evaluation

Ein Embedding-Layer befindet sich normalerweise auf der obersten Schicht eines neuronalen Netzwerks. Es wandelt die Tokens aus den Eingabedaten in Vektorrepräsentationen um. Auf der Grundlage eines Embedding-Layers simulieren die weiteren Layer die Beziehung von Eingabe und Ausgabe durch neuronale Netze. Bei vortrainiertem Embedding-Modell handelt es sich um ein veröffentlichtes Embedding-Modell, dessen Vektorrepräsentationen bereits an bestimmte Aufgaben angepasst werden, nämlich *vortrainiert*. Bei kleineren Datensätzen können seine Vektorrepräsentationen als Ausgangszustand des Embedding-Layers in das neuronale Netzwerk eingesetzt werden. Ein *Word2Vec*-Embedding repräsentiert die Tokens im Gegensatz dazu nur anhand des vorhandenen Trainingsdatensatzes. Es hat zwar den Vorteil, dass mit dem man die Anzahl der Neuronen im Embedding-Layer und die Größe des Vokabulars selbst bestimmen kann. Allerdings ist es schwer für ein *Word2Vec*-Embedding, bei kleineren Datensätzen die vielfätigen Eigenschaften jedes Tokens darzustellen. Außerdem verarbeitet *Word2Vec*-Embedding die Tokens ohne Berücksichtigung des Kontextes, d. h. Die Vektorrepräsentation des gleichen Tokens ist bei allen Kontexten immer gleich. Ein weiterer Unterschied besteht darin, wie *Word2Vec*-Embedding und vortrainiertes Embedding-Modell unbekannte Tokens behandeln. Unbekannten Tokens sind diejenige Tokens, die relativ selten im Trainingsdatensatz bzw. in den Vortrainingskorpora vorkommen und deswegen nicht im Vokabular des Embedding-Modells enthalten sind. Bei einem *Word2Vec*-Embedding werden unbekannte Tokens immer als *OOV* (Out-of-Vocabulary) behandelt. Alle *OOV* bekommen die gleiche Vektorrepräsentation. Vortrainierte Embedding-Modelle zerlegen unbekannte oder längere Tokens in Wortteilen, die im Vokabular enthalten sind, und repräsentieren solche Tokens mit den Vektorrepräsentationen ihrer (bekannten) Wortteilen. Diese Vorangehensweise für unbekannte Tokens eignet sich vermutlich besonders gut für deutsche Texte, die viele Komposita und grammatische Beugungen beinhalten.

3.1 Verwendung des BERT-Embeddings

BERT-Embedding (Bidirectional Encoder Representations from Transformers) ist aktuell ein einflussreiches vortrainiertes Embedding-Modell, das auf der Transformerarchitektur aufbaut. *German BERT Model* (bert-base-german-cased) ist die BERT-Version für die deutsche Sprache. Die Verwendung des *German BERT Models* hat das Ergebnis der Labelerkennung in diesem Teilprojekt erheblich verbessert. *Bert-Embedding* setzt einen BERT-Tokenizer voraus. Deswegen muss die Label-Token-Zuordnung den Ansprüchen des BERT-Tokenizers angepasst werden. Zwei Anpassungen werden vorgenommen. Erstens werden die Geschichten in Sätze geschnitten und in das Eingabeformat des BERT-Tokenizers überführt, weil die Transformerarchietktur von BERT nur für Eingabe bis zu zwei Sätzen geeignet ist. Zweitens werden Labels an entsprechenden Stellen der Labelsliste hinzugefügt, wenn der BERT-Tokenizer ein Token in Wortteile zerlegt, damit alle Wortteile eines Tokens das Label des Tokens übernehmen.

3.2 Evaluation

Bei der endgültigen Evaluation werden die von BERT-Tokenizer zerlegten Tokens wieder zusammengeführt, damit die Genauigkeit der Vorhersage auf der Token-Ebene bewertet wird. Die Zusammenführung der Tokens wird wie folgendes implementiert: Solange ein Wortteil des Tokens ein *Insider*-Label (alle inhaltlichen Labels) hat, trägt das Token dieses Label. Gibt es mehrere

(verschiedene) *Insider*-Labels in einem Token, trägt das Token das Label des letzten Wortteils. Die Evaluation bezieht sich selbstverständlich nur auf die *Insider*-Labels. (die 8 Hauptkategorien) d. h. Bei der Berechnung der Micro- und Macroprecision und des -recalls werden die *Outsider*- und *Padding*-Labels herausgefiltert. Allerdings ist es unvermeidlich, im Lernprozess die Verlustfunktion nach Labels der Wortteilen zu berechnen. Um das Lernergebnis bezüglich der *Insider*-Labels nach jedem Lernschritt darstellen zu können, wird eine ergänzende Metrik an der Schnittstelle von *Keras* hinzugefügt. Mit dieser Metrik werden ebenfalls die *Outsider*- und „Padding"-Labels herausgefiltert. Der Unterschied zwischen der ergänzenden Metrik und der endgültigen Evaluation besteht nur darin, dass bei der Evaluation während des Lernprozesses die Wortteilen nicht zusammengeführt werden. Die Evaluation ohne Zusammenführung der Wortteilen wrid folgendes als Evaluation auf der *Subtoken*-Ebene bezeichnet. Daten in den Abbildungen, die den Lernprozess nach Epochenzahl zeigen, werden auf der *Subtoken*-Ebene berechnet. Zwischen den Ergebnissen der beiden Evaluationsansätze (auf der Token- und Subtoken-Ebene) gibt es in der Regel keinen erheblichen Unterschied. Eine lockere Evaluation, bei der eine gewisse Abweichung bei der Labelsposition zugelassen wird, könnte dazu beitragen, einen höheren Recall (Anteil der abgerufenen Instanzen) zu erreichen. Allerdings nähert sich die Labelerkennung mit lockerer Evaluation dem Teilprojekt I (Klassifikation von kurzen Textsegmenten) an. Eine lockere Evaluation wird deswegen nicht implementiert, damit das zweite Ziel einer Labelerkennung (**Wo** sind die Labels?) nicht verwässert wird.

4 Modellarchitektur

Die spärliche Verteilung der Labels im Datensatz ist eine Herausforderung für die Labelerkennung. Nach der Label-Token-Zuordnung sind nur 1,32% der Tokens im Datensatz als sexuell motivierte Ausdrücke gekennzeichnet. Alle anderen Tokens bekommen immer das *Outsider*-Label in der Label-Token-Zuordnung. Wegen dieses unausgewogenen Insider-Outsider-Verhältnisses wird die Verlustfunktion hauptsächlich von den *Outsider*-Labels, die die absolute Mehrheit ausmachen, bestimmt, wenn alle Labels im Datensatz gleich behandelt werden. Um das Problem umzugehen werden zwei Maßnahmen benutzt. Zuerst wird der Datensatz intensiviert, d. h. nur diejenige Texte, die zumindest ein *Insider*-Label beinhalten, sind in den intensivierten Datensatz einbezogen. Die folgende Tabelle vergleicht die Anzahl der Texte aus jeder Studie im vollständigen und intensivierten Datensatz.

Anzahl von Geschichten	Studie1	Studie2	Studie3	Studie4	Studie5	Studie6
vollständig	687	902	1417	1367	2809	1739
intensiviert	348	610	872	569	856	1157

Tabelle 4: Anzahl der Geschichten im vollständgen und intensivierten Datensatz

Durchschnittlich gegen 40% aller Texten aus dem vollständigen Datensatz sind in den intensivierten Datensatz einbezogen. Alle Labels sind vollständig erhalten. Drei Viertel der Texte aus jeder Studie (nach der Intensivierung) werden als Trainingsdatensatz verwendet. Das andere Viertel wird als Testingsdatensatz verwendet.

Außerdem übernimmt die Verlustfunktion eine angepasste Klassengewichtungsvariable (*class-weight*), d. h. bei der Berechnung der Verlustfunktion wird der Verlust von jedem Label mit einer entsprechenden Gewichtung multipliziert. Fehler bei den *Insider*-Labels führen wegen einer höheren

Gewichtung zu größerem Verlust als Fehler bei *Outsider*-Labels. *Padding*-Labels bekommen eine Null-Gewichtung. Dadurch ist der Fokus der Vorhersage von den *Outsider*- und *Padding*-Labels auf den *Insider*-Labels verschoben. Im Teilprojekt 1 wird eine höhere Gewichtung den seltener vorkommenden Subkategorien erteilt, damit das Modell mehr Wert darauf legt, die selteneren Labels vorherzusagen. Aber in diesem Teilprojekt bekommen alle *Insider*-Labels die gleiche Gewichtung, weil für die Labelerkennung alle *Insider*-Labels von gleicher Bedeutung sind. Tabelle 5 zeigt die Menge der *Insider*-Labels im Testingsdatensatz)

Label	Instrumentelle	Sonstige	Zielerreichung	Erfolgserwartung
Menge	1753	1101	473	408

Label	Bedürfnis	Affektlage	Hindernisse	Misserfolgserwartung
Menge	319	304	118	44

Tabelle 5: Menge der „Insider“-Labels im Testingsdatensatz

4.1 Optimierung der Klassengewichtung

Die Klassengewichtung hat einen entscheidenden Einfluss auf die Leistung der neuronalen Netzwerke. Um eine optimale Klassengewichtung herauszufinden, werden folgende Varianten mit gleicher Modellarchitektur (vortrainiertes Embedding-Layer + Bidirectional LSTM-Layer + TimeDistributed Output-Layer mit „softmax“-activation) durchprobiert.

Bezeichnung	Alle Insider	Outsider	Padding
weight1	1	1	0
weight5	5	1	0
weight10	10	1	0
weight15	15	1	0
weight20	20	1	0
weight-balanced	26.7	1	0
weight30	30	1	0

Tabelle 6: Varianten von Klassengewichtungen

Im intensivierten Datensatz beträgt die Menge der *Outsider*-Labels das 26.7-fache der *Insider*-Labels. Deswegen ist die Gewichtung für *Insider*-Labels in der *weight-balanced* Option auf 26.7 festgelegt. Als das Modell den höchsten F1-Score bei der Vorhersage der *Insider*-Labels im Testingsdatensatz (Evaluation auf der Subtoken-Ebene) erreicht, gilt es als der beste Zustand des Modells. Abbildung 1 zeigt die Leistung der in Tabelle 6 aufgelisteten Klassengewichtungen bei ihren jeweiligen besten Zuständen und die gebrauchte Epochenzahl, um die besten Zustände zu erreichen.

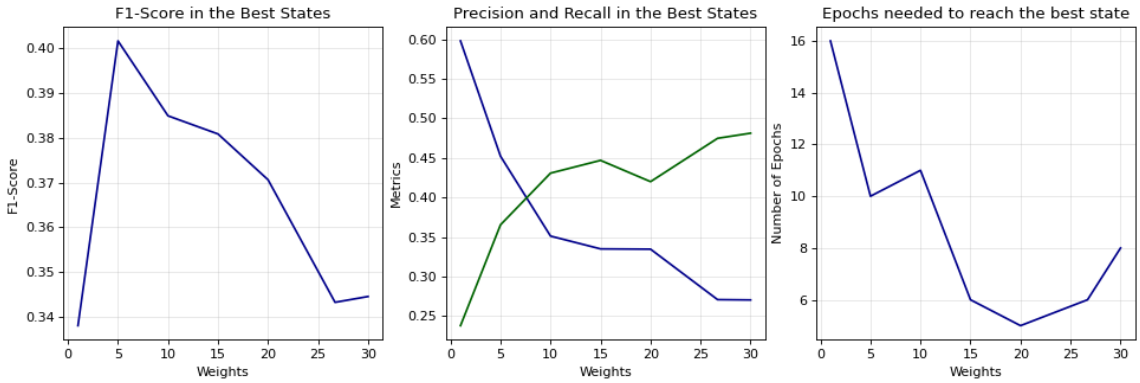


Abbildung 1: Evaluation unterschiedlicher Klassengewichtungen

Aus der Abbildung 1 lassen sich folgende Schlussfolgerungen ziehen:

1) Die Klassengewichtung „weight5“ hat den höchsten F1-Score von 40,2% erreicht.

2) Bei größerer Gewichtungsungleichheit erreicht das Modell höheren Recall auf Kosten der Precision. Zwischen 5 und 10 liegen Precision und Recall relativ nahe beieinander.

3) Das Modell braucht bei größerer Gewichtung für *Insider*-Labels insgesamt wenigere Epochen, um den besten Zustand zu erreichen. Wenn die Gewichtung 20 überholt, nimmt die Anzahl der gebrauchten Epochen wieder zu. Grund dafür liegt vermutlich darin, dass übermäßige Gewichtungsungleichheit das Modell verwirren könnte.

4.2 Experimenten mit verschiedenen Modellarchitekturen

Bezeichnung	Typ des Layers	Shape	Vermerk
Input-Layer	InputLayer	(batch, max-len)	max-len=173
Embedding	Embedding	(batch, max-len, embedding-dim)	embedding-dim=768
Hidden-Layer(s)	Dense, LSTM, GRU	(batch, max-len, units)	s. Tabelle 8
Output-Layer	TimeDistributed	(batch, max-len, len(tag2idx))	len(tag2idx)=10

Tabelle 7: Grundarchitektur

Bemerkungen:

1) „max-len“ ist die maximale Anzahl der Wortteilen in einem Eingabesatz. Paddings werden am Ende der kürzeren Sätze eingefügt.

2) BERT-German hat eine Embeddingdimension von 768 Neuren in jedem Layer.

3) In den „Hidden-Layer(s)“ werden verschiedene Arten von neuronalen Netzen (vor allem rekurrente Layer) experimentiert.

4) TimeDistributed-Outputlayer verwendet „softmax“-Aktivierungsfunktion.

5) Optimizer des Modells: „adam“, Verlustfunktion: „categorical-crossentropy“

Die Klassengewichtung „weight5“ wird in allen weiteren Experimenten verwendet. Alle in den Experimenten verwendete Modelle bauen auf einer gemeinsamen Grundarchitektur auf. Tabelle 7 skizziert die Struktur dieser Grundarchitektur. Zwischen dem Embedding-Layer und dem Output-Layer werden unterschiedliche Hidden-Layer eingesetzt. Alle Modelle teilen den gemeinsamen Input-Layer, vortrainierten Embedding-Layer und Output-Layer mit den in der Tabelle 7 beschriebenen Einstellungen.

Tabelle 8 beschreibt die in den Experimenten untersuchten Hidden-Layer. Bei den ersten vier Varianten wird das Hidden-Layer mit einem Dense-Layer oder einem rekurrenten Layer (LSTM und GRU) mit Bidirectional-Wrapper erfüllt. Bei den letzten beiden Varianten besteht das Hidden-Layer aus zwei rekurrenten Layer. Um Überanpassungen wegen tieferer Modellarchitektur zu verringern, wird ein Dropout-Layer, das in jeder Epoche zufällig 20% der Daten entfernt, nach jedem rekurrenten Layer in den letzten beiden Varianten hinzugefügt.

Bezeichnung	Typ des Layers	Units	Dropout	Vermerk
SimpleDense	Dense	128	0	„relu“ activation
BLSTM-60	Bidirectional LSTM	60*2	0	
BLSTM-173	Bidirectional LSTM	173*2	0	
BGRU-173	Bidirectional GRU	173*2	0	
BLSTM+LSTM	Bidirectional LSTM, LSTM	173*2, 173	0.2, 0.2	2 Hidden-Layer
BGRU+GRU	Bidirectional GRU, GRU	173*2, 173	0.2, 0.2	2 Hidden-Layer

Tabelle 8: Hidden-Layer

Die Ergebnisse der verschiedenen Hidden-Layer sind in der Abbildung 2 zusammengefasst. Als Metrikenansätze sind Precision, Recall und F1-Score der Vorhersage der *Insider*-Labels im Testingsdatensatz ausgewählt. Der beste Zustand eines Modells wird auch nach F1-Score im Testingsdatensatz definiert. Spezifische Lernprozesse und Überanpassungen (Differenz im Trainings- und Testingsdatensatz) der Modellen sind im Anhang II und III zu finden.

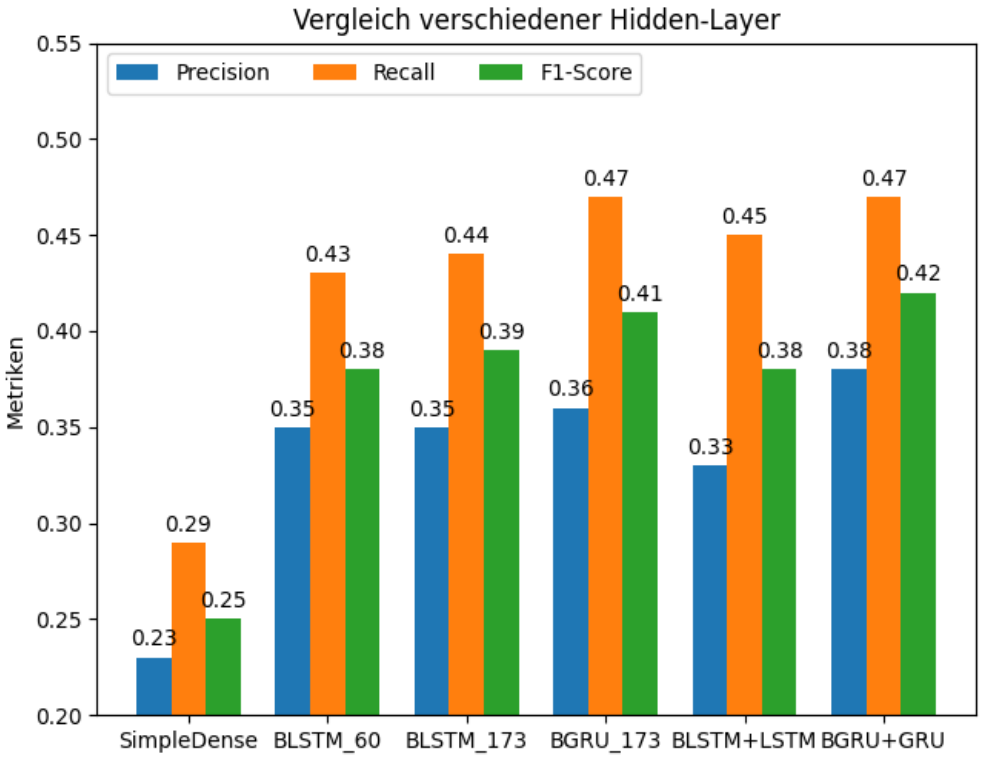


Abbildung 2: Vergleich verschiedener Hidden-Layer (Evaluation auf der Token-Ebene)

Durch einem Vergleich der Leistungen lässt sich Folgendes feststellen:

1) Alle Modelle mit rekurrentem Layer haben deutlich bessere Ergebnisse als Modell mit einem Dense-Layer erreicht.

2) Wenn man „BLSTM-173“ mit „BGRU-173“ bzw. „BLSTM+LSTM“ mit „BGRU+GRU“ vergleicht, überholt Modell mit GRU-Layer Modell mit LSTM-Layer um 2- bis 4%.

3) Modelle mit zwei rekurrenten Layers (BLSTM+LSTM, BGRU+GRU) haben keinen deutlichen Vorteil gegen den einfacheren Modellen (BLSTM-173, BGRU-173).

4) Wenn man die Überanpassung der Modelle im Anhang III betrachtet, ist das Modell mit „GRU-173“ zu bevorzugen. Nach Metriken im Testingsdatensatz liegt „GRU-173“ mit kleinem Abstand nach „BGRU+GRU“ auf dem zweiten Platz. Allerdings ist die Überanpassung an den Trainingsdatensatz von „GRU-173“ deutlich niedriger als „BGRU+GRU“.

Tabelle 9 zeigt das spezifische Ergebnis vom Modell mit „GRU-173“ Hidden-Layer bei jedem Label. Alle Daten sind auf der Token-Ebene berechnet. Es ist offensichtlich, je häufiger ein Label im Datensatz vorkommt, desto besser kann es im Allgemeine vom Modell erkannt werden. Am deutlichsten spiegelt sich diese Regel beim Recall im Testingsdatensatz. Das Label *Hindernisse* lässt sich besonders schwer zu erkennen. Andere Modellarchitekturen haben auch kein besseres Ergebnis bei diesem Label erzielt. Eine gleiche Klassengewichtung für alle *Insider*-Labels hat sowohl Vorteil als auch Nachteil. Precision bei den mesiten Labels liegen wegen der gleicher Gewichtung relativ nahe beieinander. (von 29% bis zu 39% außer bei dem Label *Hindernisse*) Allerdings lassen sich seltenere Labels schwer abrufen, da ihnen keine höhere Gewichtung erteilt wurde.

Label	Precision-Test	Recall-Test	F1-Test	Support-Test	F1-Train	Support-Train
Instrumentelle	0.38	0.57	0.45	1753	0.51	5015
Sonsitige	0.39	0.50	0.44	1101	0.53	3289
Zielerreichung	0.38	0.47	0.42	473	0.49	1411
Erfolgserwartung	0.29	0.35	0.32	408	0.45	1086
Bedürfnis	0.38	0.39	0.38	319	0.38	905
Affektlage	0.29	0.22	0.25	304	0.46	817
Hindernisse	**0.00**	**0.00**	**0.00**	118	**0.03**	312
Misserfolgserwartung	0.31	0.09	0.14	44	0.22	114
Macro-Durchschnitt	0.30	0.32	0.30	4520	0.38	12949
Micro-Durchschnitt	**0.36**	**0.47**	**0.41**	4520	**0.49**	12949

Tabelle 9: Spezifisches Ergebnis des Modells mit einem GRU Hidden Layer

4.3 Fazit

Wenn man dieses Teilprojekts mit dem Teilprojekts I vergleicht, kann man grob sagen, dass mit Maschinellem Lernverfahren vier Fünfte aller Labels sich richtig klassifizieren lässt (81% Genauigkeit im Teilprojekt I) und die Hälfte davon an den exakten Stellen erkannt werden kann. Das „BGRU-173“-Modell mit einem 41% F1-Score reicht zwar von Weitem für eine automatische Annotation nicht

aus, hat aber schon das Ergebnis der anfänglichen Versuchen verdreifacht. Zur Verbesserung der Modelllesitung hat das *BERT-German* vortrainiertes Embedding-Modell die wichtigste Rolle gespielt. Da die tieferen Modellarchitekturen kaum besseres Ergebnis als die Modelle mit einem Hidden-Layer erreicht haben, haben die Modelle sich vermutlich nur auf den oberflächlichen Merkmalen der Texte konzentriert. Andere Embedding-Modelle, lockere Evaluation und eine Optimierung der Parameter des neuronalen Netzwerks sind für die weiteren Untersuchungen empfehlenswert.

Anhang I: Lernprozesse verschiedener Klassengewichtungen

(Evaluation auf der Subtoken-Ebene, Definition s. 3.2 Evaluation)

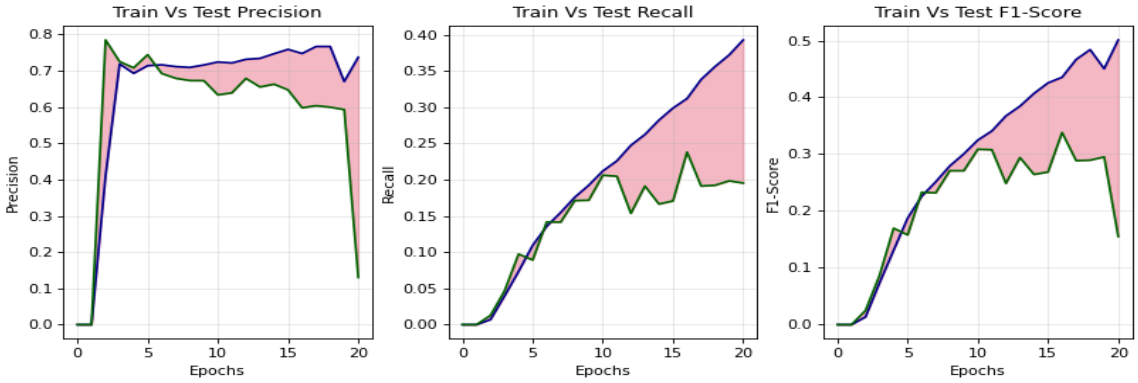


Abbildung 3: weight1: bester Zustand nach 16 Epochen



Abbildung 4: weight5: bester Zustand nach 10 Epochen



Abbildung 5: weight10: bester Zustand nach 11 Epochen



Abbildung 6: weight15: bester Zustand nach 6 Epochen

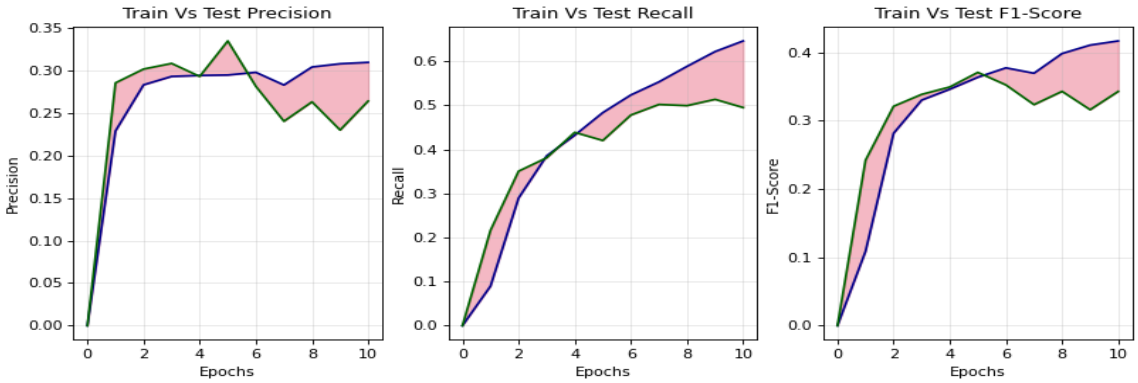


Abbildung 7: weight20: bester Zustand nach 5 Epochen

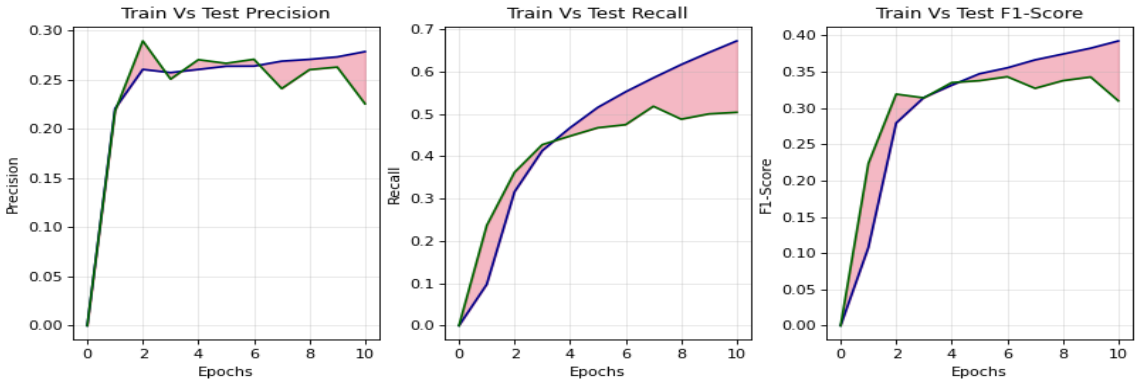


Abbildung 8: weight-balanced: (26.7) bester Zustand nach 6 Epochen

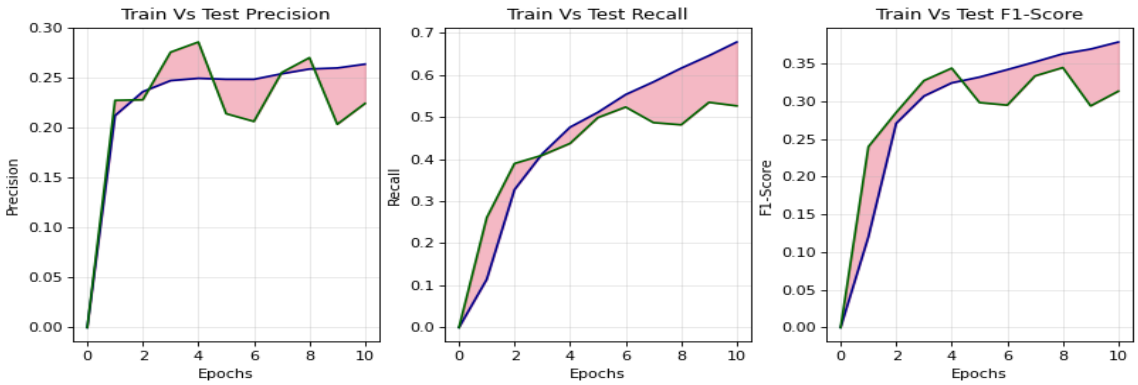


Abbildung 9: weight30: bester Zustand nach 8 Epochen

Anhang II: Lernprozesse der Modelle mit verschiedenen Hidden Layer
(Evaluation auf der Subtoken-Ebene, Definition s. 3.2 Evaluation)

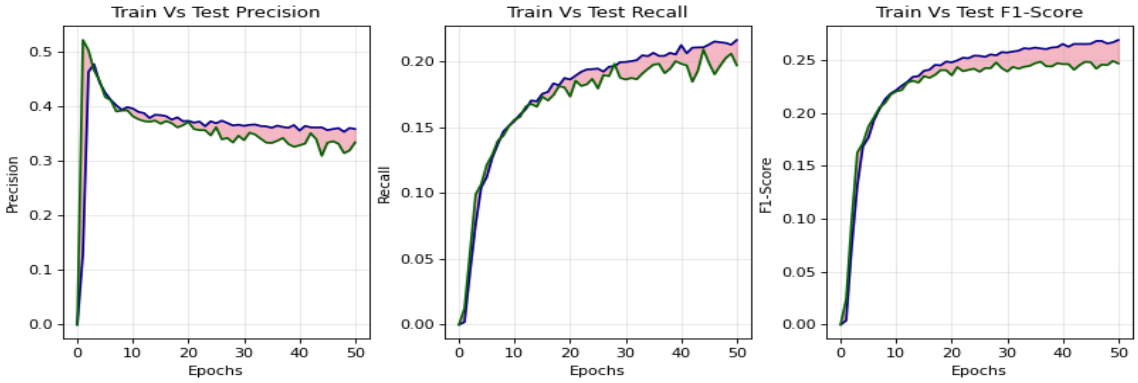


Abbildung 10: SimpleDense: bester Zustand nach 49 Epochen



Abbildung 11: BLSTM-60: bester Zustand nach 13 Epochen



Abbildung 12: BLSTM-173: bester Zustand nach 10 Epochen



Abbildung 13: BGRU-173: bester Zustand nach 12 Epochen



Abbildung 14: BLSTM+LSTM: bester Zustand nach 10 Epochen

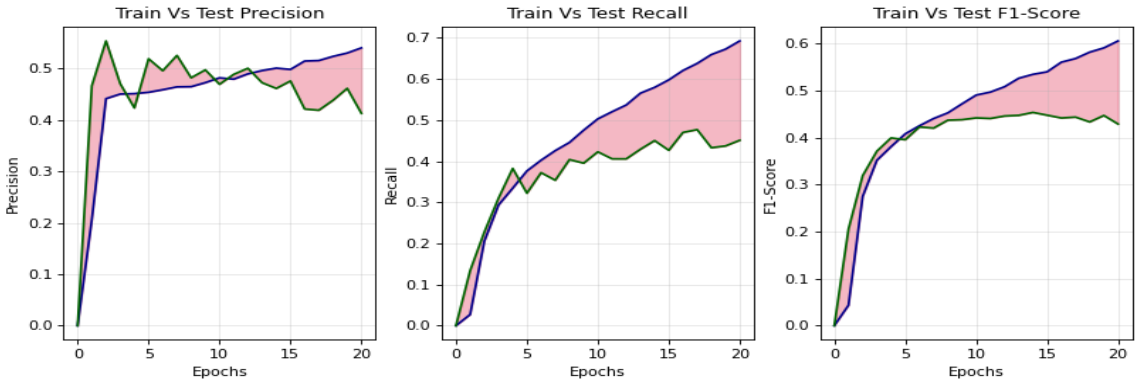


Abbildung 15: BGRU+GRU: bester Zustand nach 14 Epochen

Anhang III: Überanpassung verschiedener Modelle

(Evaluation auf der Token-Ebene, Definition s. 3.2 Evaluation)

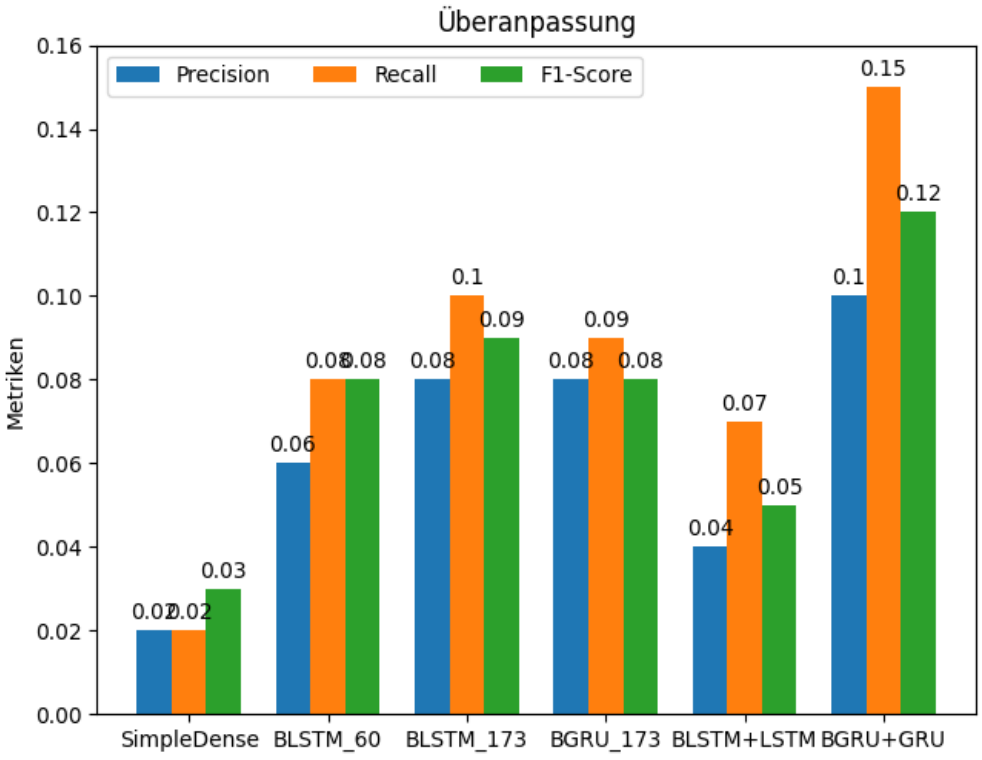


Abbildung 16: Überanpassung verschiedener Modelle

Versicherung der selbstständigen Anfertigung

Der Unterzeichnete versichert, dass er die vorliegende schriftliche Hausarbeit selbstständig verfasst und keine anderen als die von ihm angegebenen Hilfsmittel benutzt hat. Die Stellen der Arbeit, die anderen Werken dem Wortlaut oder dem Sinne nach entnommen sind, wurden in jedem Fall unter Angabe der Quellen (einschließlich des World Wide Web und anderer elektronischer Text- und Datensammlungen) kenntlich gemacht. Dies gilt auch für beigegebene Zeichnungen, bildliche Darstellungen, Skizzen und dergleichen.

Nürnberg, 6. März 2024
Unterschrift des Verfassers der Seminararbeit:
Xinyao Lu